Algorithm for file updates in Python

Project description

In this project, I created a Python algorithm that automatically updates a file containing a list of approved IP addresses. This allow list is used by a healthcare company to control which employees can access restricted patient information. The algorithm removes IP addresses from the allow list when they appear in a remove list. This process ensures that only authorized users have network access.

Open the file that contains the allow list

In this step, the `open()` function is used with a **with statement** to safely open the file named `"allow_list.txt"`.
The `with` keyword automatically manages file access and closes the file when done.
Example:

```
with open(import_file, "r") as file:
```

Here, `"r"` means the file is opened in **read mode**, and `file` is a variable that temporarily holds the opened file.

Read the file contents

The `.read()` method reads the entire contents of the file and converts it into a single string.
Example:

```
ip_addresses = file.read()
```

This allows the program to handle all IP addresses as one string before processing them further.

Convert the string into a list

The `.split()` method is used to separate the string into individual elements (IP addresses), forming a list.

```
ip_addresses = ip_addresses.split()
```

Each IP address becomes an item in the list, making it easier to compare and modify.

Iterate through the remove list

A `for` loop is used to check each element of the `ip_addresses` list.
Example:

```
for element in ip_addresses:
```

This loop lets the algorithm compare each IP address with the remove list to decide which should be deleted.

Remove IP addresses that are on the remove list

Inside the loop, an `if` conditional checks whether an element from `ip_addresses` is also in the `remove_list`.
If it is, the `.remove()` method deletes it.
Example:

```
if element in remove_list:
        ip_addresses.remove(element)
```

This ensures that only allowed IPs remain.

Update the file with the revised list of IP addresses

Update the file with the revised list of IP addresses

After updating the list, the program joins it back into a single string using `" ".join()`. Then it opens the file again in **write mode ("w")** to overwrite the old content.
Example:

```
with open(import_file, "w") as file:

        file.write(ip_addresses)
```

This step finalizes the update to the allow list.

Summary

This algorithm automates the process of maintaining an allow list by removing IP addresses that no longer have access permissions. It demonstrates the use of key Python concepts, including the `open()` function, file handling with the `with` statement, string and list methods (`.read()`, `.split()`, `.join()`), and iteration with `for` loops. The algorithm also uses conditional logic and the `.remove()` method to ensure data accuracy. Overall, it provides an efficient way to manage user access to secure network resources.

COMPLETE ALGORITHM

```
# Define a function named `update_file` that takes in two parameters: `import_file` and `remove_list`
# and combines the steps you've written in this lab leading up to this

def update_file(import_file, remove_list):

  # Build `with` statement to read in the initial contents of the file

  with open(import_file, "r") as file:

        # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

        ip_addresses = file.read()

  # Use `.split()` to convert `ip_addresses` from a string to a list

  ip_addresses = ip_addresses.split()

  # Build iterative statement
  # Name loop variable `element`
  # Loop through `ip_addresses`

  for element in ip_addresses:

        # Build conditional statement
        # If current element is in `remove_list`,
```

```
        if element in remove_list:
        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)
  # Convert `ip_addresses` back to a string so that it can be written into the text file
  ip_addresses = " ".join(ip_addresses)
  # Build `with` statement to rewrite the original file
  with open(import_file, "w") as file:
        # Rewrite the file, replacing its contents with `ip_addresses`
        file.write(ip_addresses)
# Call `update_file()` and pass in "allow_list.txt" and a list of IP addresses to be removed
update_file("allow_list.txt",["192.168.69.116","192.168.156.224"])
# Build `with` statement to read in the updated file
with open("allow_list.txt", "r") as file:
  # Read in the updated file and store the contents in `text`
  text = file.read()
# Display the contents of `text`
print(text)
```

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [22]:
```python
# Define a function named `update_file` that takes in two parameters: `import_file` and `remove_list`
# and combines the steps you've written in this lab leading up to this

def update_file(import_file, remove_list):

  # Build `with` statement to read in the initial contents of the file

  with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

    ip_addresses = file.read()

  # Use `.split()` to convert `ip_addresses` from a string to a list

  ip_addresses = ip_addresses.split()

  # Build iterative statement
  # Name loop variable `element`
  # Loop through `ip_addresses`

  for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

      # then current element should be removed from `ip_addresses`

      ip_addresses.remove(element)

  # Convert `ip_addresses` back to a string so that it can be written into the text file

  ip_addresses = " ".join(ip_addresses)

  # Build `with` statement to rewrite the original file

  with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)

# Call `update_file()` and pass in "allow_list.txt" and a list of IP addresses to be removed

update_file("allow_list.txt",["192.168.69.116","192.168.156.224"])

# Build `with` statement to read in the updated file

with open("allow_list.txt", "r") as file:

  # Read in the updated file and store the contents in `text`

  text = file.read()

# Display the contents of `text`

print(text)
```

```
192.168.25.60 192.168.6.9 192.168.133.188 192.168.218.219
```